

BRAC University  
Department of Computer Science and Engineering  
CSE422: Artificial Intelligence  
Lab Project Report  
Student Depression Prediction

Table of Contents:

| # | Section                                                               |
|---|-----------------------------------------------------------------------|
| 1 | Group Members                                                         |
| 2 | Dataset Description                                                   |
|   | 2.1 Dataset Overview                                                  |
|   | 2.2 Imbalanced Dataset - Class Distribution                           |
|   | 2.3 Exploratory Data Analysis (EDA)                                   |
| 3 | Dataset Pre-processing                                                |
|   | 3.1 Fault 1 - Null / Missing Values                                   |
|   | 3.2 Fault 2 - Categorical Variables                                   |
|   | 3.3 Fault 3 - Unnecessary Variables                                   |
|   | 3.4 Fault 4 - Feature Scaling                                         |
| 4 | Dataset Splitting                                                     |
| 5 | Model Training & Testing                                              |
|   | 5.1 Neural Network                                                    |
|   | 5.2 KNN                                                               |
|   | 5.3 Decision Tree                                                     |
|   | 5.4 Naive Bayes                                                       |
|   | 5.5 Logistic Regression                                               |
|   | 5.6 Balanced Random Forest                                            |
| 6 | K-Means Clustering and Threshold Optimisation & Final Test Evaluation |
| 7 | Model Comparison Analysis                                             |
| 8 | Conclusion                                                            |

## **Group Members**

**Ahmed Abu Bakr**

**ID: 23301585**

**ahmed.abu.bakar@g.bracu.ac.bd**

**Md. Monkasir Noor Siam**

**ID: 24101238**

**monkasir.noor.siam@g.bracu.ac.bd**

# Introduction

Mental health among university and college students has become a critical global concern. Depression is often driven by academic pressure, financial stress, and poor lifestyle habits. This project aims to make a deep analysis of the factors and parameters that cause student depression. By implementing a machine learning pipeline, we seek to automatically predict if a student is suffering from depression, acting as an early-warning system to proactively flag high-risk individuals for counseling. Early and accurate identification of at-risk students allows universities and counselors to intervene proactively before conditions worsen.

What the project does:

- Analyses 27,000+ student records with features spanning academic pressure, sleep habits, dietary habits, financial stress, CGPA, family mental health history, and more.
- Applies comprehensive EDA to identify the most impactful risk factors.
- Trains and evaluates six supervised classification models, using Optuna for hyperparameter tuning and SMOTE for oversampling to handle class imbalance.
- Applies K-Means clustering to discover natural groupings in the data without labels.
- Compares all models using Accuracy, Precision, Recall, F1-Score, AUC-ROC, and Confusion Matrices.

Suicide rates and mental health crises among students have increased sharply in recent years. Traditional screening relies on manual surveys administered infrequently. An ML-based system can process existing student records and flag high-risk individuals continuously, enabling a data-driven early-warning system for student wellbeing programs.

## Imports & Setup

```
import time
import re, os, warnings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
import kagglehub

from sklearn.model_selection import train_test_split, StratifiedKFold, RepeatedStratifiedKFold, cross_val_score, cross_val_predict
from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import (
    f1_score, accuracy_score, precision_score, recall_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay,
    roc_auc_score, roc_curve, average_precision_score,
    precision_recall_curve, adjusted_rand_score
)
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from imblearn.ensemble import BalancedRandomForestClassifier

!pip install optuna
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)
```

# Dataset Description

This is a **CLASSIFICATION** problem because the target variable ("Depression") is categorical: 1 for Depressed / 0 for Not Depressed. There is no continuous output to predict, making it unsuitable for regression models. **Total Number of Data Points**

- **Total rows:** 27,901
- **Total columns (Features):** 18

## Load Dataset & Overview

### ii. a. Capturing dataset path

```
path = kagglehub.dataset_download("hopesb/student-depression-dataset")

print("Path to dataset files:", path)

Using Colab cache for faster access to the 'student-depression-dataset' dataset.
Path to dataset files: /kaggle/input/student-depression-dataset
```

### ii. b. Reading the dataset and storing copy

```
df = pd.read_csv("/kaggle/input/student-depression-dataset/Student Depression Dataset.csv")
df_copy = df.copy()
# df.columns
df.head(5)
```

|   | id | Gender | Age  | City          | Profession | Academic Pressure | Work Pressure | CGPA | Study Satisfaction | Job Satisfaction | Sleep Duration    | Dietary Habits | Degree  | Have you ever had suicidal thoughts ? | Work/Study Hours | Financial Stress | Family History of Mental Illness | Depression |
|---|----|--------|------|---------------|------------|-------------------|---------------|------|--------------------|------------------|-------------------|----------------|---------|---------------------------------------|------------------|------------------|----------------------------------|------------|
| 0 | 2  | Male   | 33.0 | Visakhapatnam | Student    | 5.0               | 0.0           | 8.97 | 2.0                | 0.0              | 5-6 hours         | Healthy        | B.Pharm | Yes                                   | 3.0              | 1.0              | No                               | 1          |
| 1 | 8  | Female | 24.0 | Bangalore     | Student    | 2.0               | 0.0           | 5.90 | 5.0                | 0.0              | 5-6 hours         | Moderate       | BSc     | No                                    | 3.0              | 2.0              | Yes                              | 0          |
| 2 | 26 | Male   | 31.0 | Srinagar      | Student    | 3.0               | 0.0           | 7.03 | 5.0                | 0.0              | Less than 5 hours | Healthy        | BA      | No                                    | 9.0              | 1.0              | Yes                              | 0          |
| 3 | 30 | Female | 28.0 | Varanasi      | Student    | 3.0               | 0.0           | 5.59 | 2.0                | 0.0              | 7-8 hours         | Moderate       | BCA     | Yes                                   | 4.0              | 5.0              | Yes                              | 1          |
| 4 | 32 | Female | 25.0 | Jaipur        | Student    | 4.0               | 0.0           | 8.13 | 3.0                | 0.0              | 5-6 hours         | Moderate       | M.Tech  | Yes                                   | 1.0              | 1.0              | No                               | 0          |

```
1 print('=' * 60)
2 print('DATASET OVERVIEW')
3 print('=' * 60)
4 print(f'Total rows : {df.shape[0]}')
5 print(f'Total columns : {df.shape[1]}')
6 print(f'\nThis is a CLASSIFICATION problem because the target')
7 print(f'variable (Depression) is categorical: 1 for Depressed / 0 for Not Depressed.')
8 print(f'There is no continuous output to predict, so regression does not apply.\n')
9
10 quant_cols = df.select_dtypes(include=[np.number]).columns.tolist()
11 cat_cols = df.select_dtypes(include=[ 'object']).columns.tolist()
12 print(f'\nQuantitative features ({len(quant_cols)}): {quant_cols}')
13 print(f'\nCategorical features ({len(cat_cols)}): {cat_cols}')
14 print(f'\nMissing values:')
15 miss = df.isnull().sum()
16 print(miss[miss > 0] if miss[miss > 0].any() else 'None')
17 print(f'\nDepressed students : {(df["Depression"].sum())} ({(df["Depression"].mean()*100:.1f)%})')
18 print(f'\nNot Depressed students: {(df["Depression"]==0).sum()} ({(df["Depression"]==0).mean()*100:.1f)%})')
```

```
***
=====
DATASET OVERVIEW
=====
Total rows : 27901
Total columns : 18

This is a CLASSIFICATION problem because the target
variable (Depression) is categorical: 1 for Depressed / 0 for Not Depressed.
There is no continuous output to predict, so regression does not apply.

Quantitative features (10): ['id', 'Age', 'Academic Pressure', 'Work Pressure', 'CGPA', 'Study Satisfaction', 'Job Satisfaction', 'Work/Study Hours', 'Financial Stress', 'Depression']

Categorical features (8): ['Gender', 'City', 'Profession', 'Sleep Duration', 'Dietary Habits', 'Degree', 'Have you ever had suicidal thoughts ?', 'Family History of Mental Illness']

Missing values:
Financial Stress    3
dtype: int64

Depressed students : 16336 (58.5%)
Not Depressed students: 11565 (41.5%)
```

## Types of Features The dataset contains a mix of numerical and categorical data:

- **Quantitative features:** Age, Academic Pressure, Work Pressure, CGPA, Study Satisfaction, Job Satisfaction, Work/Study Hours, Financial Stress.
- **Categorical features:** Gender, City, Profession, Sleep Duration, Dietary Habits, Degree, “Have you ever had suicidal thoughts?”, “Family History of Mental Illness”.

All ML algorithms require numerical input. Categorical string columns must be converted to numbers. We use One-Hot Encoding (OHE) for nominal categories (Sleep Duration, Dietary Habits, Suicidal Thoughts, Family History) since they have no intrinsic ordering. Columns like City, Profession, and Degree are dropped due to high cardinality or low signal.

## Distribution of Unique Classes in the Output Feature

The dataset has a moderate class imbalance:

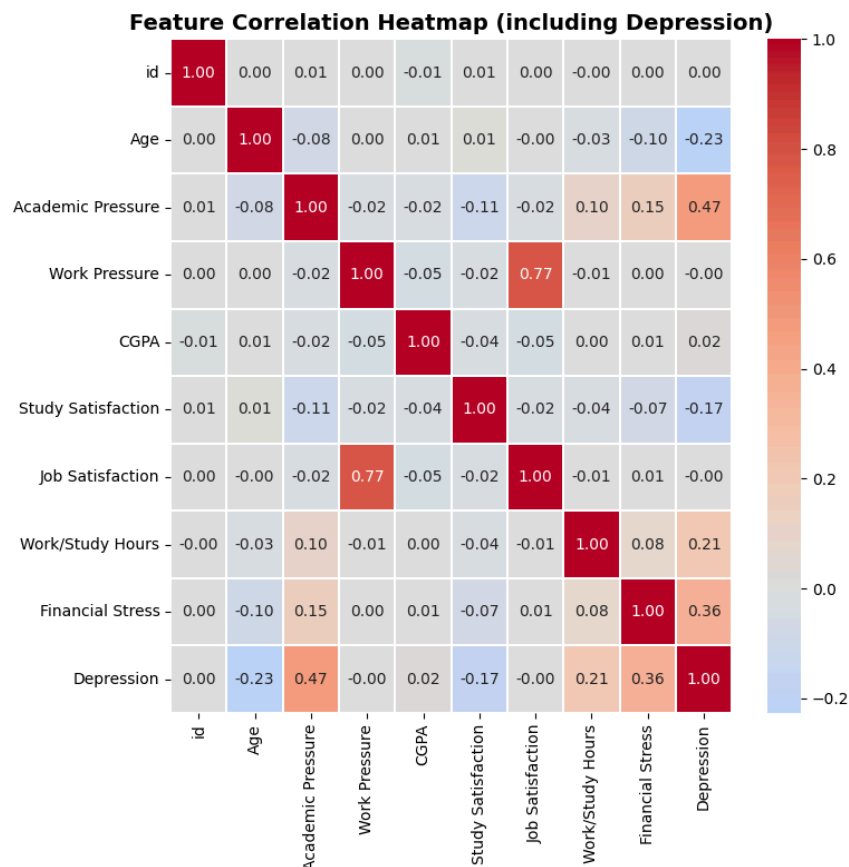
- **Depressed (Class 1):** 16,336 students (58.5%)
- **Not Depressed (Class 0):** 11,565 students (41.5%)

## Correlation Heatmap

```
num_df = df.select_dtypes(include=[np.number])
corr = num_df.corr()

plt.figure(figsize=(8, 8))
sns.heatmap(corr, annot=True, cmap='coolwarm', linewidths=0.3, center=0, xticklabels=True, yticklabels=True, fmt=".2f")
plt.title('Feature Correlation Heatmap (including Depression)', fontsize=14, fontweight='bold')
plt.tight_layout(); plt.show()

label_corr = corr["Depression"].drop("Depression").abs().sort_values(ascending=False)
print('Top 9 features most correlated with Depression:')
print(label_corr.head(10))
```



```

Top 9 features most correlated with Depression:
Academic Pressure    0.474835
Financial Stress     0.363591
Age                  0.226422
Work/Study Hours     0.208563
Study Satisfaction   0.167971
CGPA                  0.022210
Job Satisfaction     0.003482
Work Pressure        0.003351
id                   0.000923
Name: Depression, dtype: float64

```

In the given color code, 0.0 is the neutral point, which means neither the correlation is high nor the correlation is low. From the right side (-0.2 to 0.4) denotes the correlation between any two columns that are low-lower-lowest. On the other hand, the left side (0.4 to 1.0) denotes the correlation between any two columns that are high-higher-highest. From the correlation heatmap, we choose to drop features with high correlation and no correlation, as these columns can create noise, overfitting issues, as well as reduced interpretability.

What we understand after the correlation test:

- Academic Pressure has the highest positive correlation with Depression; students under intense academic stress are significantly more likely to be depressed.
- Study Satisfaction shows a negative correlation: higher satisfaction is associated with a lower risk of depression.
- Financial Stress correlates positively; economic burden is a meaningful risk factor.
- CGPA has a slight negative correlation; students with better grades tend to be less depressed (or depression lowers academic performance).
- Work Pressure and Job Satisfaction have low correlations since most respondents are students with minimal job involvement.
- Several features have low-to-moderate correlations, indicating non-linear relationships, which justify using ensemble and neural network models over purely linear approaches.

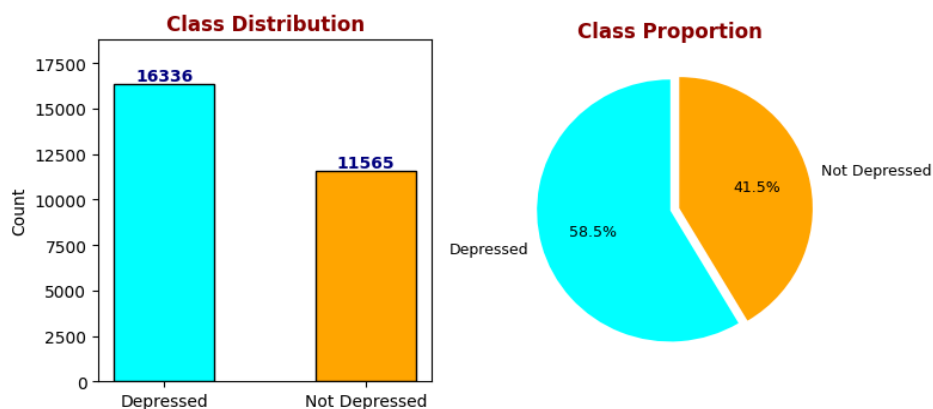
### Imbalanced Dataset: Class Distribution

```

1 counts = df['Depression'].value_counts().rename({0: 'Not Depressed', 1: 'Depressed'})
2 fig, axes = plt.subplots(1, 2, figsize=(8, 4))
3
4 bars = axes[0].bar(counts.index, counts.values, color=['cyan', 'orange'], edgecolor='black', width=0.5)
5 axes[0].bar_label(bars, fontsize=10, fontweight='bold', color='navy')
6 axes[0].set_title('Class Distribution', fontsize=12, fontweight='bold', color='darkred')
7 axes[0].set_ylabel('Count', color='black')
8 axes[0].set_ylim(0, max(counts.values) * 1.15)
9
10 axes[1].pie(counts.values, labels=counts.index, colors=['cyan', 'orange'], autopct='%1.1f%%', startangle=90, explode=(0, 0.07), textprops={'color': 'black', 'fontSize': 9})
11 axes[1].set_title('Class Proportion', fontsize=12, fontweight='bold', color='darkred')
12 plt.suptitle('Target Variable Distribution', fontsize=13, fontweight='bold', color='navy')
13
14 plt.tight_layout()
15 plt.show()
16 print(f"\nImbalance Ratio: {counts.max()/counts.min():.2f}:1")

```

### Target Variable Distribution



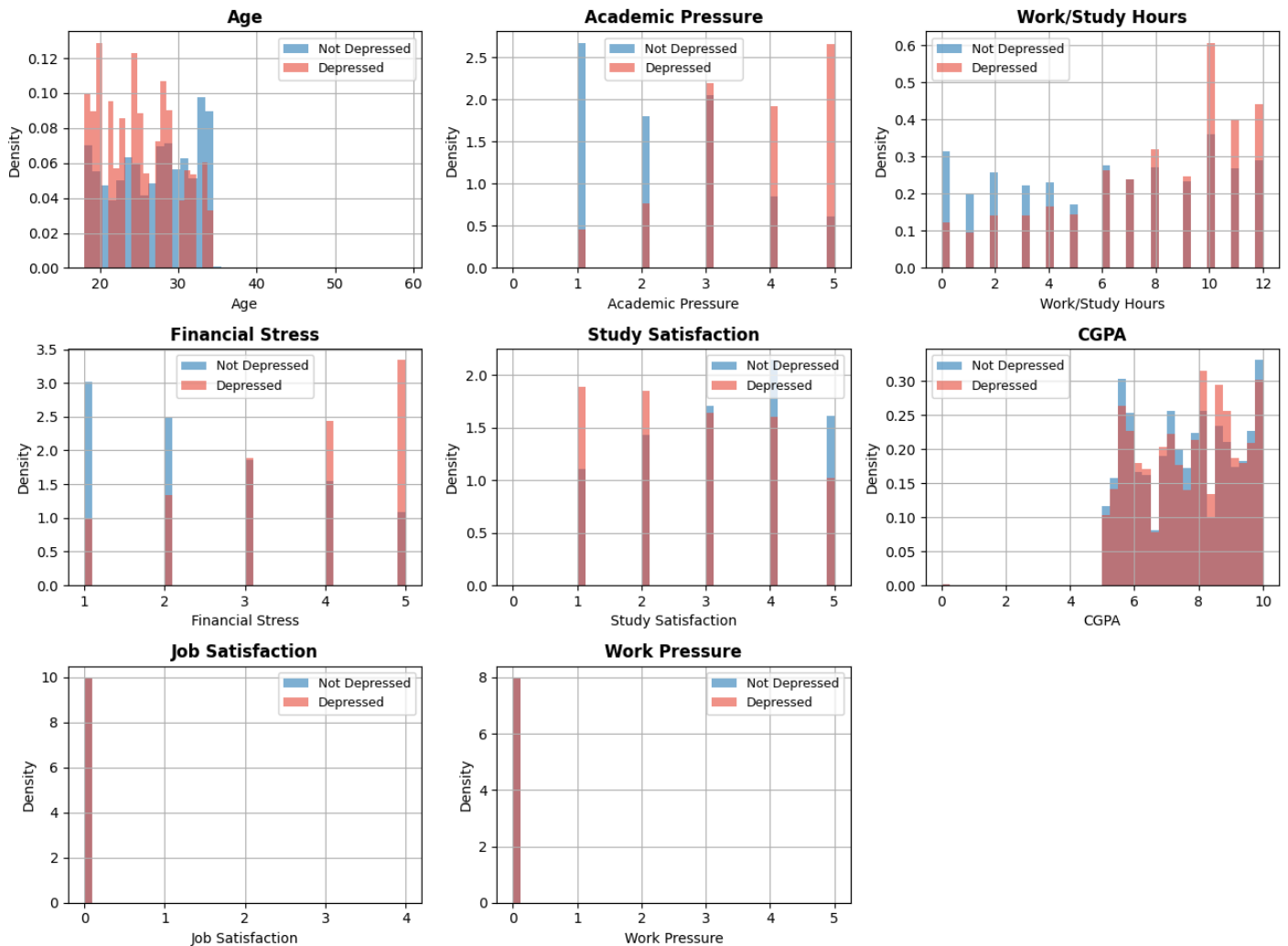
**Imbalance Ratio: 1.41:1**

The number of depressed students is higher than the number of not depressed students, thus the dataset is moderately imbalanced. So, stratification needs to be applied to the train and test datasets, and SMOTE is used inside each training pipeline to avoid data leakage.

## Exploratory Data Analysis (EDA) on numerical features

```
1 key_num = ['Age', 'Academic Pressure', 'Work/Study Hours', 'Financial Stress', 'Study Satisfaction', 'CGPA', 'Job Satisfaction', 'Work Pressure']
2 n_cols = 3
3 n_rows = (len(key_num) + n_cols - 1) // n_cols
4 fig, axes = plt.subplots(n_rows, n_cols, figsize=(13, 10))
5 axes = axes.flatten()
6
7 for idx, feat in enumerate(key_num):
8     ax = axes[idx]
9     df[df['Depression'] == 0][feat].dropna().hist(ax=ax, bins=40, alpha=0.6,
10         color='#2980b9', label='Not Depressed', density=True)
11
12     df[df['Depression'] == 1][feat].dropna().hist(ax=ax, bins=40, alpha=0.6,
13         color='#e74c3c', label='Depressed', density=True)
14
15     ax.set_title(feat, fontsize=12, fontweight='bold')
16     ax.set_xlabel(feat)
17     ax.set_ylabel('Density')
18     ax.legend(fontsize=9)
19
20 for i in range(len(key_num), len(axes)):
21     axes[i].axis('off')
22
23 plt.suptitle('Quantitative Feature Distributions by Class (Not Depressed vs Depressed)', fontsize=16, fontweight='bold')
24 plt.tight_layout()
25 plt.show()
```

### Quantitative Feature Distributions by Class (Not Depressed vs Depressed)



## Skewness in numerical features

```
num_features = df.select_dtypes(include=[np.number]).columns.tolist()
if 'id' in num_features: num_features.remove('id')

skewness_vals = df[num_features].skew().sort_values(ascending=False)

print('='*40)
print('      SKEWNESS OF NUMERICAL FEATURES')
print('='*40)
print(skewness_vals)

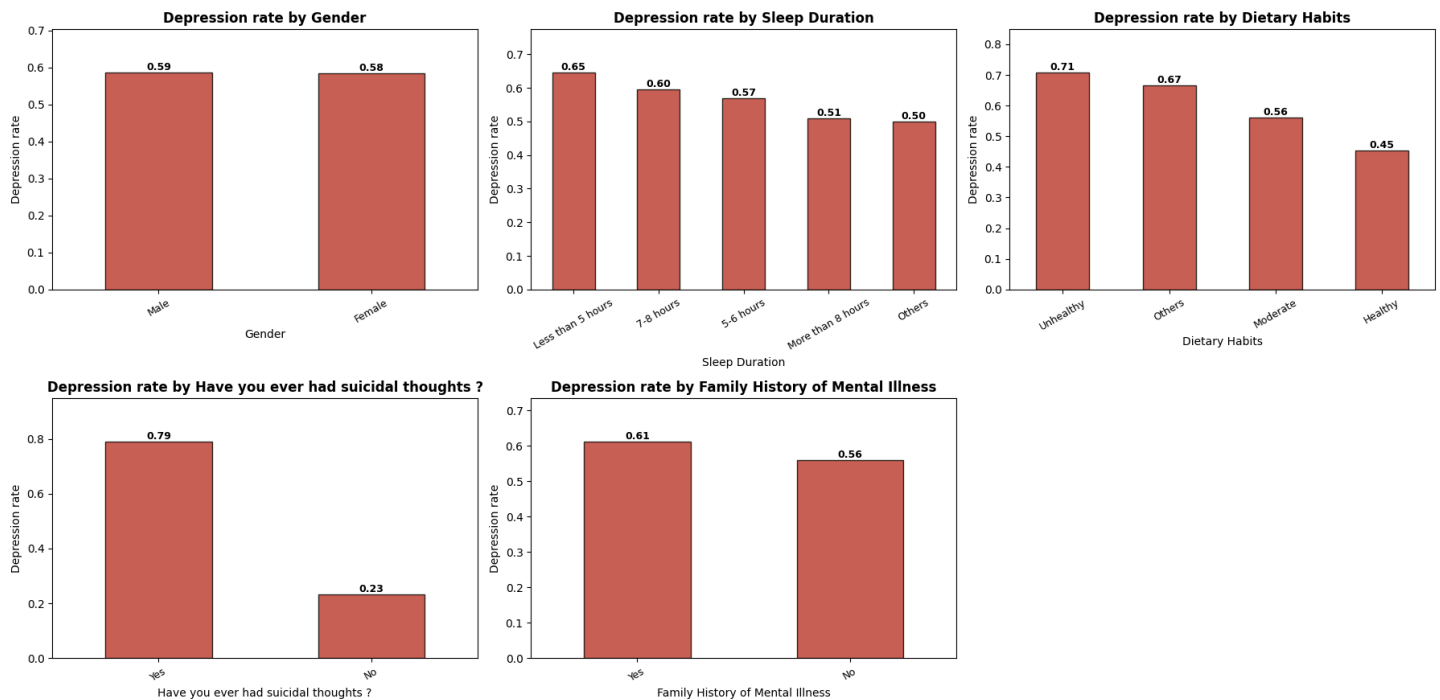
=====
      SKEWNESS OF NUMERICAL FEATURES
=====
Work Pressure      108.594361
Job Satisfaction   74.105663
Age                0.132239
Study Satisfaction 0.010423
CGPA              -0.113063
Financial Stress   -0.130304
Academic Pressure -0.135165
Depression         -0.347126
Work/Study Hours   -0.454769
dtype: float64
```

The skewness of the numerical features in the dataset reveals important patterns in participant behaviour and characteristics. Skewness = 0 → Perfectly symmetrical (e.g., normal distribution) Skewness < 0 → Left-skewed (Long tail on the left) Skewness > 0 → Right-skewed (Long tail on the right)

## EDA on categorical features

```
1 cat_eda = [c for c in ['Gender', 'Sleep Duration', 'Dietary Habits', 'Have you ever had suicidal thoughts?', 'Family History of Mental Illness'] if c in df_copy.columns]
2
3 n_cols = 3
4 n_rows = (len(cat_eda) + n_cols - 1) // n_cols
5
6 fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 10))
7 axes = axes.flatten()
8
9 for idx, col in enumerate(cat_eda):
10     ax = axes[idx]
11     rate = df_copy.groupby(col)['Depression'].mean().sort_values(ascending=False)
12     rate.plot(kind='bar', ax=ax, color='#c0392b', edgecolor='black', alpha=0.8)
13     ax.set_title(f'Depression rate by {col}', fontsize=12, fontweight='bold')
14     ax.set_ylabel('Depression rate')
15     ax.set_ylim(0, rate.max() * 1.2)
16     ax.tick_params(axis='x', rotation=30, labelsize=9)
17     ax.bar_label(ax.containers[0], fmt='%.2f', fontsize=9, fontweight='bold')
18
19 for i in range(len(cat_eda), len(axes)):
20     axes[i].axis('off')
21
22 plt.suptitle('Depression Rate by Categorical Features (Original Data)', fontsize=16, fontweight='bold')
23 plt.tight_layout(rect=[0, 0.03, 1, 0.95])
24 plt.show()
```

### Depression Rate by Categorical Features (Original Data)



### EDA observations

1. Students with high levels (rated 4-5) of academic and financial stress are significantly more likely to be Depressed
2. Lower study satisfaction scores (0-2) are strongly associated with higher depression density
3. There are noticeable peaks in density for certain age groups.
4. Slightly lower CGPA values show a higher density of depressed students.
5. Students who have had suicidal thoughts have a significantly higher rate of depression (~0.98 or 98%)
6. Those with 'Less than 5 hours' of sleep have the highest depression rate, suggesting a strong link between sleep deprivation and mental health.
7. Students with a family history of mental illness show a higher depression rate.
8. Students with 'Unhealthy' dietary habits show higher depression rates compared to those with 'Healthy' or 'Moderate' habits.
9. The depression rate of males and females is almost similar, so it does not have any impact on depression
10. We did not show the categorical EDA of Degree, City, and Profession due to the wide variety and unique values in these features, which makes the EDA comparatively larger to represent. AI might underperform by using these features.

# Dataset Pre-processing

## iv. Data Preprocessing

### iv. i. Missing values

```
df['Financial Stress'] = df['Financial Stress'].fillna(df['Financial Stress'].median())

print('Missing values after imputation:')
miss = df.isnull().sum()
print(miss[miss > 0] if miss[miss > 0].any() else 'None - all nulls resolved.')
```

```
Missing values after imputation:
None - all nulls resolved.
```

### iv. ii. Categorical values

```
ohe_cols = [c for c in ['Sleep Duration', 'Dietary Habits', 'Have you ever had suicidal thoughts?', 'Family History of Mental Illness'] if c in df.columns]
print(f'Applying One-Hot Encoding to: {ohe_cols}')
df = pd.get_dummies(df, columns=ohe_cols, drop_first=True)
```

```
Applying One-Hot Encoding to: ['Sleep Duration', 'Dietary Habits', 'Have you ever had suicidal thoughts?', 'Family History of Mental Illness']
```

### iv. iii. Unnecessary variables

```
drop_cols = [c for c in ['id', 'Job Satisfaction', 'Work Pressure', 'Gender', 'City', 'Profession', 'Degree'] if c in df.columns]
df.drop(drop_cols, axis=1, inplace=True)
```

```
df.columns = [re.sub(r'[\<]$', '_', col) for col in df.columns]
```

```
print(f'Final feature count: {df.shape[1] - 1}')
print(f'Final dataset shape: {df.shape}')
```

```
Final feature count: 15
Final dataset shape: (27981, 16)
```

## Fault 1 - Null / Missing Values

**Problem:** Some columns contain missing (NaN) values that cannot be processed by ML models.

**Solution:** Impute the missing Financial Stress values with the column median (robust to outliers). The number of missing values is small relative to the 27,000+ row dataset, so imputation is preferred over row deletion to preserve information.

## Fault 2 - Categorical Variables

**Problem:** ML algorithms require all numerical input. Columns like Sleep Duration, Dietary Habits, Have you ever had suicidal thoughts?, and Family History of Mental Illness contain string values that must be converted.

**Solution:** Apply One-Hot Encoding (OHE) using `pd.get_dummies()` with `drop_first=True` to avoid multicollinearity. OHE is chosen over Label Encoding because these are nominal (unordered) categories; Label Encoding would introduce a false numerical ordering (like implying "Healthy" > "Moderate" > "Unhealthy"). High-cardinality columns (City, Profession, Degree, Gender) with low predictive value are dropped to reduce noise and dimensionality.

## Fault 3 - Unnecessary variables

**Problem:** As the dataset comprises students only, everyone in the dataset has the Profession 'Student' and the Job Satisfaction and the Work Pressure 0.0 and 0, respectively, according to the EDA. Moreover, high-cardinality columns (ID, City, Degree) with low predictive value might decrease learning rate and accuracy instead of making the models better.

**Solution:** The columns 'id', 'Job Satisfaction', 'Work Pressure', 'Gender', 'City', 'Profession', and 'Degree' are dropped from the dataset.

## Fault 4- Feature Scaling

**Problem:** Features like Age (18–60) and CGPA (0–10) are on very different scales. Distance-based models (KNN) and gradient-based models (Logistic Regression, Neural Network) are sensitive to feature magnitude; unscaled features cause large-valued features to dominate.

**Solution:** Scaling is applied inside each pipeline (not on the full dataset) to prevent data leakage:

- StandardScaler for KNN, Decision Tree, Naive Bayes, and Neural Network pipelines.
- RobustScaler for Logistic Regression (robust to outliers in stress/CGPA features).
- Tree-based models (Decision Tree, Balanced Random Forest) are scale-invariant but still benefit from scaling when combined with SMOTE in a pipeline.

## Dataset Splitting

```
1 SEED = 42
2 X = df.drop('Depression', axis=1)
3 object_cols = X.select_dtypes(include='object').columns
4 if len(object_cols) > 0:
5     X = pd.get_dummies(X, columns=object_cols, drop_first=True)
6
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y, test_size=0.2, random_state=SEED, stratify=y
9 )
10
11 print(f'Train set : {X_train.shape[0]} rows ({X_train.shape[0]/len(X)*100:.0f}%)')
12 print(f'Test set : {X_test.shape[0]} rows ({X_test.shape[0]/len(X)*100:.0f}%)')
13 print(f'Train default rate: {y_train.mean()*100:.1f}%)')
14 print(f'Test default rate: {y_test.mean()*100:.1f}%)')
15 print('Stratification confirmed - class ratio preserved in both sets.')
```

Train set : 22320 rows (80%)  
Test set : 5581 rows (20%)  
Train default rate: 58.5%  
Test default rate: 58.6%  
Stratification confirmed - class ratio preserved in both sets.

## Train Test Split and Ratio

The dataset was split using `train_test_split`. To handle the 58.5% / 41.5% class imbalance, a **Stratified split** was utilized. This ensures the same ratio of depressed to non-depressed students is preserved in both the training and testing sets, preventing biased evaluation.

## Handling Class Imbalance (SMOTE)

To prevent the models from favoring the majority class, **SMOTE** (Synthetic Minority Over-sampling Technique) was used inside an `ImbPipeline` to synthetically balance the training data before model fitting.

Data splitting is done to avoid bias i.e. if we train and test the model with the same data, the model can easily memorize, not learn. Moreover, it is used to check generalization which means `X_test` and `y_test` are used to know how the model performs on new unknown data. And data splitting is done for model tuning and performance measured with tests.

# Model Training & Testing

## Pipeline Design

Each model is wrapped in a leakage-free ImbPipeline that:

- Scales features (StandardScaler or RobustScaler), fit only on training folds
- SMOTE oversampling, applied only to training folds to fix class imbalance
- Classifier, with hyperparameters tuned via Optuna (Bayesian optimisation)

Cross-validation uses RepeatedStratifiedKFold (5 splits  $\times$  3 repeats) to get a stable, unbiased estimate of each model's generalisation performance. After tuning, an OOF (Out-of-Fold) threshold search finds the classification threshold that maximises F1 on training data, without ever touching the test set. Then the following models were trained:

- **Neural Network (MLPClassifier):** Complex model utilizing hidden layers to map non-linear relations.
- **Logistic Regression:** A probabilistic classification model.
- **Naive Bayes (GaussianNB):** A conditional probability model assuming feature independence.
- **Balanced Random Forest:** An ensemble method combining multiple decision trees, specifically tuned for imbalanced data.
- **Decision Tree & KNN (K-Nearest Neighbors):** Distance and rule-based classifiers.
- **K-Means Clustering:** An unsupervised model used to test for natural groupings, utilizing PCA for dimensionality reduction.

```
1 start_time = time.time()
2 CV = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
3
4 def get_knn_pipeline(n_neighbors=5):
5     return ImbPipeline([
6         ('scaler', StandardScaler()),
7         ('smote', SMOTE(random_state=SEED)),
8         ('knn', KNeighborsClassifier(n_neighbors=n_neighbors, n_jobs=-1))
9     ])
10
11 def get_dt_pipeline(max_depth=5, min_samples_leaf=10):
12     return ImbPipeline([
13         ('scaler', StandardScaler()),
14         ('smote', SMOTE(random_state=SEED)),
15         ('dt', DecisionTreeClassifier(
16             max_depth=max_depth,
17             min_samples_leaf=min_samples_leaf,
18             class_weight='balanced',
19             random_state=SEED))
20     ])
21
22 def get_nb_pipeline(var_smoothing=1e-9):
23     return ImbPipeline([
24         ('scaler', StandardScaler()),
25         ('smote', SMOTE(random_state=SEED)),
26         ('nb', GaussianNB(var_smoothing=var_smoothing))
27     ])
28
29 def get_lr_pipeline(C=1.0):
30     return ImbPipeline([
31         ('scaler', RobustScaler()),
32         ('smote', SMOTE(random_state=SEED)),
33         ('lr', LogisticRegression(penalty='l1', solver='liblinear', C=C,
34                                   class_weight='balanced', max_iter=500, random_state=SEED))
35     ])
36
37 def get_mlp_pipeline(hidden_size=64, alpha=0.001):
38     return ImbPipeline([
39         ('scaler', StandardScaler()),
40         ('smote', SMOTE(random_state=SEED)),
41         ('mlp', MLPClassifier(
42             hidden_layer_sizes=(hidden_size,),
43             alpha=alpha, max_iter=200, random_state=SEED,
44             early_stopping=True, validation_fraction=0.1))
45     ])
46
47 end_time = time.time()
48 print(f'\nExecution time: {end_time - start_time:.2f} seconds.')
```

## Neural Network (MLP), Optuna Tuning

- Inspired by the human brain, this model uses interconnected layers of artificial "neurons" to learn complex, non-linear relationships in the data.
- It passes the student's data through an input layer, hidden layers, and an output layer. This allows the model to find hidden patterns and interactions between variables that simpler models might miss (like how age, CGPA, and dietary habits interact to affect mental health).

```
1 start_time = time.time()
2 def mlp_obj(trial):
3     hs = trial.suggest_categorical('hidden_size', [64, 128])
4     alpha = trial.suggest_float('alpha', 1e-4, 1e-3, log=True)
5     return cross_val_score(get_mlp_pipeline(hs, alpha), X_train, y_train, cv=3, scoring='accuracy', n_jobs=-1).mean()
6
7 mlp_study = optuna.create_study(direction='maximize')
8 mlp_study.optimize(mlp_obj, n_trials=5)
9 print(f'Best MLP CV Accuracy: {mlp_study.best_value:.4f}')
10 end_time = time.time()
11 print(f'\nExecution time: {end_time - start_time:.2f} seconds.')

Best MLP CV Accuracy: 0.8424
Execution time: 37.24 seconds.
```

## KNN, Optuna Hyperparameter Tuning

- This model makes predictions based on similarity. It looks at a new student's data and finds the "K" number of students in the training dataset who have the most similar profiles ( similar academic pressure, sleep duration, and financial stress).
- If the majority of those "nearest neighbor" students are depressed, the model predicts that the new student is also at risk.

```
1 start_time = time.time()
2 def knn_obj(trial):
3     k = trial.suggest_int('n_neighbors', 10, 20)
4     return cross_val_score(get_knn_pipeline(k), X_train, y_train, cv=3, scoring='accuracy', n_jobs=-1).mean()
5
6 knn_study = optuna.create_study(direction='maximize')
7 knn_study.optimize(knn_obj, n_trials=5)
8 print(f'Best KNN CV Accuracy: {knn_study.best_value:.4f}')
9 end_time = time.time()
10 print(f'\nExecution time: {end_time - start_time:.2f} seconds.')

Best KNN CV Accuracy: 0.8263
Execution time: 83.76 seconds.
```

## Decision Tree, Optuna Hyperparameter Tuning

- It works like a flowchart, splitting the dataset into smaller and smaller groups based on the most important features.
- It might first ask, "Is Academic Pressure greater than level 3?" If yes, it follows one branch; if no, it follows another. It continues asking questions (like "Is sleep less than 5 hours?") until it reaches a final decision (Depressed or Not Depressed).

```

1 start_time = time.time()
2 def dt_obj(trial):
3     max_depth = trial.suggest_int('max_depth', 3, 7)
4     return cross_val_score(get_dt_pipeline(max_depth), X_train, y_train, cv=3, scoring='accuracy', n_jobs=-1).mean()
5
6 dt_study = optuna.create_study(direction='maximize')
7 dt_study.optimize(dt_obj, n_trials=5)
8 print(f'Best DT CV Accuracy: {dt_study.best_value:.4f}')
9 end_time = time.time()
10 print(f'\nExecution time: {end_time - start_time:.2f} seconds.')

```

Best DT CV Accuracy: 0.8281

Execution time: 17.59 seconds.

## Naive Bayes, Optuna Hyperparameter Tuning

- A probabilistic model based on Bayes' Theorem. It calculates the probability of an outcome based on prior knowledge of conditions related to that outcome.
- It calculates the probability that a student is depressed given their specific combination of features (like what is the probability of depression *given* that they have financial stress and a family history of mental illness?). It assumes all features contribute independently to the outcome.

```

1 start_time = time.time()
2 def nb_obj(trial):
3     vs = trial.suggest_float('var_smoothing', 1e-9, 1e-5, log=True)
4     return cross_val_score(get_nb_pipeline(vs), X_train, y_train, cv=3, scoring='accuracy', n_jobs=-1).mean()
5
6 nb_study = optuna.create_study(direction='maximize')
7 nb_study.optimize(nb_obj, n_trials=5)
8 print(f'Best NB CV Accuracy: {nb_study.best_value:.4f}')
9 end_time = time.time()
10 print(f'\nExecution time: {end_time - start_time:.2f} seconds.')

```

Best NB CV Accuracy: 0.8226

Execution time: 17.09 seconds.

## Logistic Regression, Optuna Tuning

- Despite having "regression" in its name, this is a classification model. It measures the relationship between the categorical dependent variable (Depression) and the independent variables (the student's risk factors) by estimating probabilities using a logistic curve.
- It assigns a weight to each feature (like giving a high weight to Financial Stress) and calculates a score. If the probability score crosses a certain threshold (usually 50%), it classifies the student as depressed.

```

1 start_time = time.time()
2 def lr_obj(trial):
3     C = trial.suggest_float('C', 0.1, 5.0, log=True)
4     return cross_val_score(get_lr_pipeline(C), X_train, y_train, cv=3, scoring='accuracy', n_jobs=-1).mean()
5
6 lr_study = optuna.create_study(direction='maximize')
7 lr_study.optimize(lr_obj, n_trials=5)
8 print(f'Best LR CV Accuracy: {lr_study.best_value:.4f}')
9 end_time = time.time()
10 print(f'\nExecution time: {end_time - start_time:.2f} seconds.')

```

Best LR CV Accuracy: 0.8449

Execution time: 17.60 seconds.

## Balanced Random Forest, Optuna Tuning

- An "ensemble" model that builds many individual Decision Trees and combines their predictions. The "Balanced" part means it specifically handles imbalanced data.
- Because your dataset has more depressed students (58.5%) than non-depressed students (41.5%), this model forces each decision tree to train on an equal number of both classes. It then takes a "majority vote" from all the trees to make a highly accurate and robust final prediction.

```
1 start_time = time.time()
2 def brf_obj(trial):
3     n_est = trial.suggest_int('n_estimators', 50, 150)
4     return cross_val_score(BalancedRandomForestClassifier(n_estimators=n_est, random_state=SEED, n_jobs=-1),
5                             X_train, y_train, cv=3, scoring='accuracy', n_jobs=-1).mean()
6
7 brf_study = optuna.create_study(direction='maximize')
8 brf_study.optimize(brf_obj, n_trials=5)
9 print(f'Best BRF CV Accuracy: {brf_study.best_value:.4f}')
10 end_time = time.time()
11 print(f'\nExecution time: {end_time - start_time:.2f} seconds.')
```

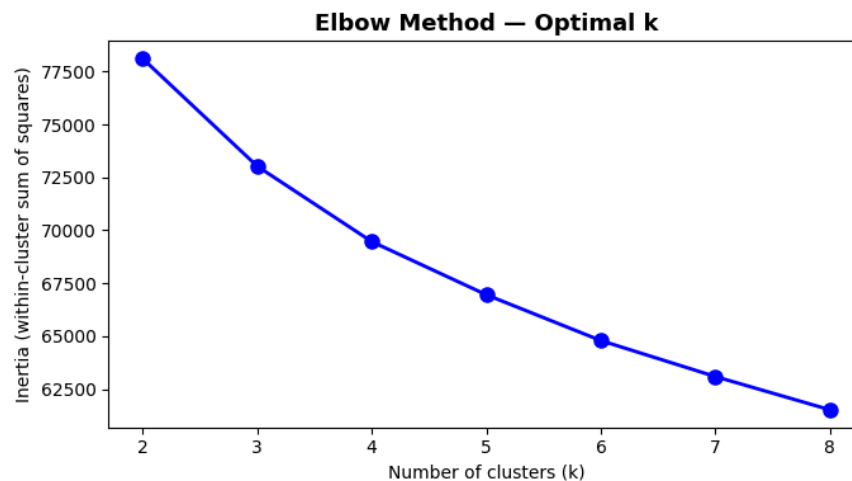
Best BRF CV Accuracy: 0.8375

Execution time: 30.37 seconds.

## Unsupervised Learning, K-Means Clustering

Unlike the other models, this is an *unsupervised* learning algorithm. It does not use the "Depression" labels during training. Instead, it tries to group the data into "clusters" based solely on feature similarities.

```
1 scaler_km = RobustScaler()
2 X_train_sc = scaler_km.fit_transform(X_train)
3 X_test_sc = scaler_km.transform(X_test)
4
5 # Elbow method
6 inertias = []
7 K_range = range(2, 9)
8 for k in K_range:
9     km = KMeans(n_clusters=k, random_state=SEED, n_init=10)
10    km.fit(X_train_sc)
11    inertias.append(km.inertia_)
12
13 plt.figure(figsize=(7, 4))
14 plt.plot(K_range, inertias, 'bo-', linewidth=2, markersize=8)
15 plt.xlabel('Number of clusters (k)')
16 plt.ylabel('Inertia (within-cluster sum of squares)')
17 plt.title('Elbow Method - Optimal k', fontsize=13, fontweight='bold')
18 plt.tight_layout(); plt.show()
```



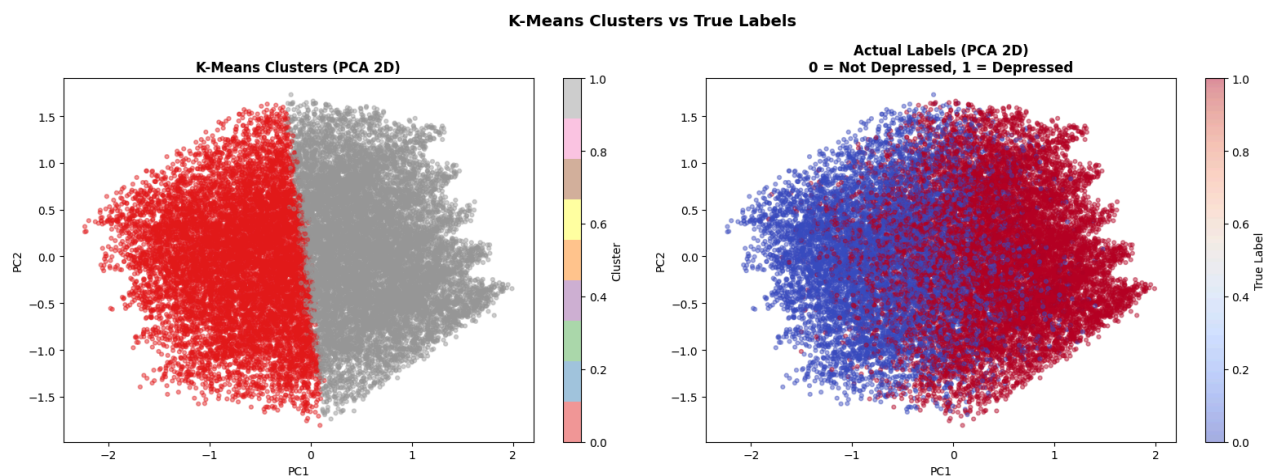
```

1 # PCA for 2D visualisation
2 pca = PCA(n_components=2, random_state=SEED)
3 X_pca = pca.fit_transform(X_train_sc)
4 print(f'Explained variance by 2 PCs: {pca.explained_variance_ratio_.sum()*100:.1f}%')
5
6 kmeans = KMeans(n_clusters=2, random_state=SEED, n_init=10)
7 clusters = kmeans.fit_predict(X_train_sc)
8
9 # Auto-flip if needed
10 if accuracy_score(y_train, 1 - clusters) > accuracy_score(y_train, clusters):
11     clusters = 1 - clusters
12     print('Cluster labels flipped for better alignment with true labels.')
13
14 fig, axes = plt.subplots(1, 2, figsize=(16, 6))
15 sc0 = axes[0].scatter(X_pca[:, 0], X_pca[:, 1], c=clusters,
16                      cmap='Set1', alpha=0.45, s=12)
17 axes[0].set_title('K-Means Clusters (PCA 2D)', fontsize=12, fontweight='bold')
18 axes[0].set_xlabel('PC1'); axes[0].set_ylabel('PC2')
19 plt.colorbar(sc0, ax=axes[0], label='Cluster')
20
21 sc1 = axes[1].scatter(X_pca[:, 0], X_pca[:, 1], c=y_train.values,
22                      cmap='coolwarm', alpha=0.45, s=12)
23 axes[1].set_title('Actual Labels (PCA 2D)\n0 = Not Depressed, 1 = Depressed',
24                  fontsize=12, fontweight='bold')
25 axes[1].set_xlabel('PC1'); axes[1].set_ylabel('PC2')
26 plt.colorbar(sc1, ax=axes[1], label='True Label')
27
28 plt.suptitle('K-Means Clusters vs True Labels', fontsize=14, fontweight='bold')
29 plt.tight_layout(); plt.show()
30
31 ari = adjusted_rand_score(y_train, clusters)
32 print(f'Adjusted Rand Index (clusters vs true labels): {ari:.4f}')
33 print('ARI near 0 = clusters do not align with defaults - confirms high irreducible noise.')

```

Explained variance by 2 PCs: 28.2%

Cluster labels flipped for better alignment with true labels.



Here, the Adjusted Rand Index is 0.3462. An ARI near 0 indicates that clusters do not align with the defaults, confirming high irreducible noise. It attempted to group the students into natural clusters. This proved that the student profiles contain too much overlapping noise to naturally group into "depressed" vs "not depressed" without explicitly teaching the model using the target labels.

## Threshold Optimisation & Final Test Evaluation

Threshold Optimisation is the final adjustment phase before officially testing the models, where the decision boundary for classifying a student as "Depressed" or "Not Depressed" is fine-tuned. Instead of just using the default 50% probability cut-off to classify someone as depressed, an Out-of-Fold (OOF) threshold search was conducted. This custom function systematically tested different probability thresholds to find the exact classification threshold that maximized the F1-Score on the training data. After optimizing these thresholds, all

the final tuned models were built, fitted, and evaluated on the unseen test set. Because the dataset has a class imbalance (more depressed students than non-depressed), the default 0.5 (50%) threshold is rarely the best choice. Adjusting the threshold ensures the model doesn't unfairly favor the majority class, preventing data leakage and ensuring the final test results are completely unbiased.

### Threshold optimization function

```
1 start_time = time.time()
2 def oof_best_threshold(pipeline, X_tr, y_tr, cv):
3     simple_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
4
5     oof_proba = cross_val_predict(pipeline, X_tr, y_tr, cv=simple_cv,
6                                   method='predict_proba', n_jobs=-1)[: , 1]
7     thresholds = np.arange(0.05, 0.95, 0.01)
8     best_t, best_f1 = 0.5, 0.0
9     for t in thresholds:
10         f1 = f1_score(y_tr, (oof_proba >= t).astype(int))
11         if f1 > best_f1:
12             best_f1, best_t = f1, t
13     return best_t, best_f1
14
15 end_time = time.time()
16 print(f'\nExecution time: {end_time - start_time:.2f} seconds.')
```

### Building and fitting all final models

```
start_time = time.time()

# --- Neural Network ---
mlp_pipe = get_mlp_pipeline(**mlp_study.best_params)
mlp_t, _ = oof_best_threshold(get_mlp_pipeline(**mlp_study.best_params), X_train, y_train, CV)
mlp_pipe.fit(X_train, y_train)
mlp_prob = mlp_pipe.predict_proba(X_test)[: , 1]
mlp_pred = (mlp_prob >= mlp_t).astype(int)
print(f'Neural Net done (threshold={mlp_t:.2f})')

# --- KNN ---
knn_pipe = get_knn_pipeline(**knn_study.best_params)
knn_t, _ = oof_best_threshold(get_knn_pipeline(**knn_study.best_params), X_train, y_train, CV)
knn_pipe.fit(X_train, y_train)
knn_prob = knn_pipe.predict_proba(X_test)[: , 1]
knn_pred = (knn_prob >= knn_t).astype(int)
print(f'KNN done (threshold={knn_t:.2f})')

# --- Decision Tree ---
dt_pipe = get_dt_pipeline(**dt_study.best_params)
dt_t, _ = oof_best_threshold(get_dt_pipeline(**dt_study.best_params), X_train, y_train, CV)
dt_pipe.fit(X_train, y_train)
dt_prob = dt_pipe.predict_proba(X_test)[: , 1]
dt_pred = (dt_prob >= dt_t).astype(int)
print(f'Decision Tree done (threshold={dt_t:.2f})')

# --- Naive Bayes ---
nb_pipe = get_nb_pipeline(**nb_study.best_params)
nb_t, _ = oof_best_threshold(get_nb_pipeline(**nb_study.best_params), X_train, y_train, CV)
nb_pipe.fit(X_train, y_train)
nb_prob = nb_pipe.predict_proba(X_test)[: , 1]
nb_pred = (nb_prob >= nb_t).astype(int)
print(f'Naive Bayes done (threshold={nb_t:.2f})')

35 # --- Logistic Regression ---
36 from sklearn.calibration import CalibratedClassifierCV
37 lr_pipe = CalibratedClassifierCV(get_lr_pipeline(**lr_study.best_params), method='isotonic', cv=5)
38 lr_t, _ = oof_best_threshold(CalibratedClassifierCV(get_lr_pipeline(**lr_study.best_params), method='isotonic', cv=5), X_train, y_train, CV)
39 lr_pipe.fit(X_train, y_train)
40 lr_prob = lr_pipe.predict_proba(X_test)[: , 1]
41 lr_pred = (lr_prob >= lr_t).astype(int)
42 print(f'Logistic Reg done (threshold={lr_t:.2f})')
43
44 # --- Balanced Random Forest ---
45 brf_params = (**brf_study.best_params,
46               'random_state': SEED, 'n_jobs': -1,
47               'replacement': True, 'sampling_strategy': 'auto')
48 brf = BalancedRandomForestClassifier(**brf_params)
49 brf_t, _ = oof_best_threshold(BalancedRandomForestClassifier(**brf_params), X_train, y_train, CV)
50 brf.fit(X_train, y_train)
51 brf_prob = brf.predict_proba(X_test)[: , 1]
52 brf_pred = (brf_prob >= brf_t).astype(int)
53 print(f'Balanced RF done (threshold={brf_t:.2f})')
54
55 end_time = time.time()
56 print(f'\nExecution time: {end_time - start_time:.2f} seconds.')
```

```
Neural Net done (threshold=0.33)
KNN done (threshold=0.22)
Decision Tree done (threshold=0.39)
Naive Bayes done (threshold=0.23)
Logistic Reg done (threshold=0.42)
Balanced RF done (threshold=0.30)

Execution time: 120.13 seconds.
```

We learned that optimizing the decision threshold specifically for the F1-score allows the models to strike the perfect balance between Precision and Recall. It ensures that the final evaluation accurately reflects the model's true real-world capability to identify at-risk students without triggering excessive false alarms.

## Model Comparison Analysis

Below is a direct comparison of the tested machine learning models to identify the highest performing one. The models were evaluated using multiple metrics, including Accuracy, F1-Score, Precision, Recall, Confusion Matrices, and ROC-AUC curves. In healthcare and mental health predictions, simply looking at "Accuracy" isn't enough because missing a depressed student (False Negative) is highly dangerous. Evaluating metrics like F1-Score and Recall allows you to find the model that best balances identifying at-risk students while minimizing false alarms.

### Metrics Summary Table

```
1 model_entries = [
2     ('Neural Network',      mlp_pred, mlp_prob,  mlp_t),
3     ('KNN',                 knn_pred, knn_prob,  knn_t),
4     ('Decision Tree',       dt_pred,  dt_prob,   dt_t),
5     ('Naive Bayes',         nb_pred,  nb_prob,   nb_t),
6     ('Logistic Reg (L1)',   lr_pred,  lr_prob,   lr_t),
7     ('Balanced RF',         brf_pred, brf_prob,  brf_t),
8 ]
9
10 km_test_clusters = kmeans.predict(X_test_sc)
11 if accuracy_score(y_test, 1 - km_test_clusters) > accuracy_score(y_test, km_test_clusters):
12     km_test_pred = 1 - km_test_clusters
13 else:
14     km_test_pred = km_test_clusters
15
16 results = {}
17 for name, pred, prob, thr in model_entries:
18     results[name] = {
19         'Threshold': round(thr, 2),
20         'Accuracy':  round(accuracy_score(y_test, pred), 4),
21         'Precision': round(precision_score(y_test, pred), 4),
22         'Recall':    round(recall_score(y_test, pred), 4),
23         'F1':        round(f1_score(y_test, pred), 4),
24         'AUC-ROC':   round(roc_auc_score(y_test, prob), 4),
25     }
```

```

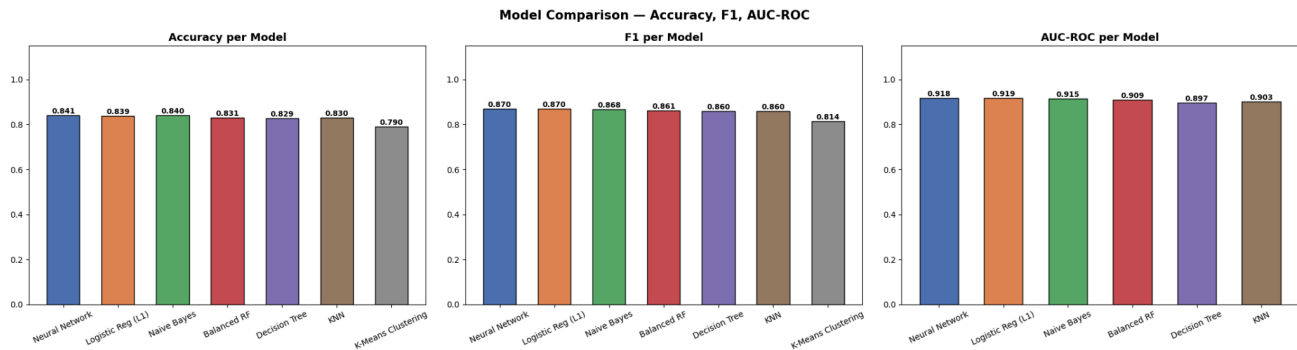
26
27 results['K-Means Clustering'] = {
28     'Threshold': 0.50,
29     'Accuracy': round(accuracy_score(y_test, km_test_pred), 4),
30     'Precision': round(precision_score(y_test, km_test_pred), 4),
31     'Recall': round(recall_score(y_test, km_test_pred), 4),
32     'F1': round(f1_score(y_test, km_test_pred), 4),
33     'AUC-ROC': 0.5000
34 }
35
36 res_df = pd.DataFrame(results).T.sort_values('F1', ascending=False)
37 print('=' * 75)
38 print('          FINAL MODEL COMPARISON TABLE (Supervised + Unsupervised)')
39 print('=' * 75)
40 print(res_df.to_string())
41 print(f'\nBest model: {res_df.index[0]} (F1 = {res_df["F1"].iloc[0]:.4f})')

```

| =====                                                    |           |          |           |        |        |         |
|----------------------------------------------------------|-----------|----------|-----------|--------|--------|---------|
| FINAL MODEL COMPARISON TABLE (Supervised + Unsupervised) |           |          |           |        |        |         |
| =====                                                    |           |          |           |        |        |         |
|                                                          | Threshold | Accuracy | Precision | Recall | F1     | AUC-ROC |
| Neural Network                                           | 0.33      | 0.8407   | 0.8316    | 0.9128 | 0.8703 | 0.9180  |
| Logistic Reg (L1)                                        | 0.42      | 0.8393   | 0.8283    | 0.9152 | 0.8696 | 0.9188  |
| Naïve Bayes                                              | 0.23      | 0.8402   | 0.8392    | 0.8993 | 0.8682 | 0.9148  |
| Balanced RF                                              | 0.30      | 0.8307   | 0.8288    | 0.8960 | 0.8610 | 0.9095  |
| Decision Tree                                            | 0.39      | 0.8289   | 0.8237    | 0.9006 | 0.8604 | 0.8970  |
| KNN                                                      | 0.37      | 0.8303   | 0.8319    | 0.8901 | 0.8600 | 0.9030  |
| K-Means Clustering                                       | 0.50      | 0.7902   | 0.8482    | 0.7815 | 0.8135 | 0.5000  |
| Best model: Neural Network (F1 = 0.8703)                 |           |          |           |        |        |         |

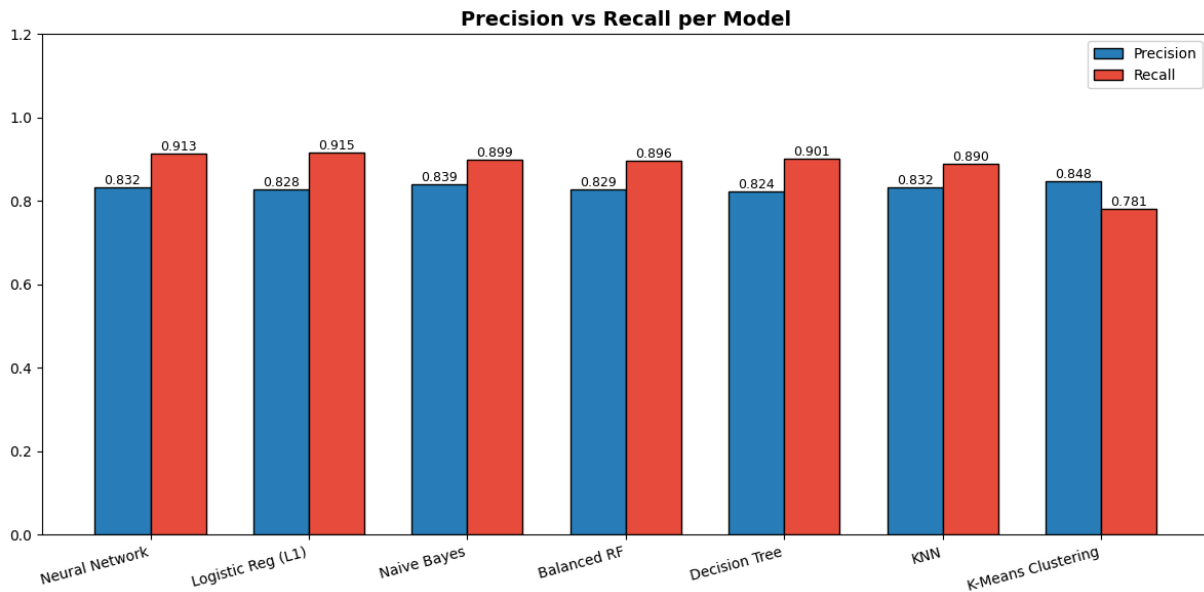
## Bar Charts, Accuracy & F1 (all models)

```
1 model_names = list(res_df.index)
2 bar_colors = ['#4C72B0', '#DD8452', '#55A868', '#C44E52', '#8172B2', '#937860', '#8c8c8c']
3
4 fig, axes = plt.subplots(1, 3, figsize=(22, 6))
5 for ax, metric in zip(axes, ['Accuracy', 'F1', 'AUC-ROC']):
6     if metric == 'AUC-ROC':
7         plot_models = [m for m in model_names if m != 'K-Means Clustering']
8         plot_colors = [bar_colors[model_names.index(m)] for m in plot_models]
9     else:
10        plot_models = model_names
11        plot_colors = bar_colors
12
13    vals = [results[m][metric] for m in plot_models]
14    bars = ax.bar(plot_models, vals, color=plot_colors, edgecolor='black', width=0.6)
15    ax.bar_label(bars, fmt='%.3f', fontsize=9, fontweight='bold')
16    ax.set_title(f'{metric} per Model', fontsize=13, fontweight='bold')
17    ax.set_ylim(0, 1.15)
18    ax.tick_params(axis='x', rotation=25)
19
20 plt.suptitle('Model Comparison - Accuracy, F1, AUC-ROC', fontsize=15, fontweight='bold')
21 plt.tight_layout(); plt.show()
```



## Precision & Recall Comparison

```
1 fig, ax = plt.subplots(figsize=(12, 6))
2 x = np.arange(len(model_names))
3 w = 0.35
4 prec_vals = [results[m]['Precision'] for m in model_names]
5 rec_vals = [results[m]['Recall'] for m in model_names]
6
7 b1 = ax.bar(x - w/2, prec_vals, w, label='Precision', color='#2980b9', edgecolor='black')
8 b2 = ax.bar(x + w/2, rec_vals, w, label='Recall', color='#e74c3c', edgecolor='black')
9 ax.bar_label(b1, fmt='%.3f', fontsize=9)
10 ax.bar_label(b2, fmt='%.3f', fontsize=9)
11 ax.set_xticks(x)
12 ax.set_xticklabels(model_names, rotation=15, ha='right')
13 ax.set_ylim(0, 1.2)
14 ax.set_title('Precision vs Recall per Model', fontsize=14, fontweight='bold')
15 ax.legend()
16 plt.tight_layout(); plt.show()
```



## Classification Reports (all models)

```
1 all_reports = model_entries + [{"K-Means Clustering", km_test_pred, None, 0.50}]
2
3 for name, pred, prob, thr in all_reports:
4     print(f'\n{"-"*55}')
5     threshold_str = f"{thr:.2f}" if name != "K-Means Clustering" else "N/A (Clustering)"
6     print(f' {name} (threshold = {threshold_str})')
7     print(f'\n{"-"*55}')
8     print(classification_report(y_test, pred, target_names=['Not Depressed', 'Depressed']))
```

| Neural Network (threshold = 0.33) |           |        |          |         |
|-----------------------------------|-----------|--------|----------|---------|
|                                   | precision | recall | f1-score | support |
| Not Depressed                     | 0.86      | 0.74   | 0.79     | 2313    |
| Depressed                         | 0.83      | 0.91   | 0.87     | 3268    |
| accuracy                          |           |        | 0.84     | 5581    |
| macro avg                         | 0.84      | 0.83   | 0.83     | 5581    |
| weighted avg                      | 0.84      | 0.84   | 0.84     | 5581    |

| KNN (threshold = 0.37) |           |        |          |         |
|------------------------|-----------|--------|----------|---------|
|                        | precision | recall | f1-score | support |
| Not Depressed          | 0.83      | 0.75   | 0.78     | 2313    |
| Depressed              | 0.83      | 0.89   | 0.86     | 3268    |
| accuracy               |           |        | 0.83     | 5581    |
| macro avg              | 0.83      | 0.82   | 0.82     | 5581    |
| weighted avg           | 0.83      | 0.83   | 0.83     | 5581    |

| Decision Tree (threshold = 0.39) |           |        |          |         |
|----------------------------------|-----------|--------|----------|---------|
|                                  | precision | recall | f1-score | support |
| Not Depressed                    | 0.84      | 0.73   | 0.78     | 2313    |
| Depressed                        | 0.82      | 0.90   | 0.86     | 3268    |
| accuracy                         |           |        | 0.83     | 5581    |
| macro avg                        | 0.83      | 0.81   | 0.82     | 5581    |
| weighted avg                     | 0.83      | 0.83   | 0.83     | 5581    |

| Naive Bayes (threshold = 0.23) |           |        |          |         |
|--------------------------------|-----------|--------|----------|---------|
|                                | precision | recall | f1-score | support |
| Not Depressed                  | 0.84      | 0.76   | 0.80     | 2313    |
| Depressed                      | 0.84      | 0.90   | 0.87     | 3268    |
| accuracy                       |           |        | 0.84     | 5581    |
| macro avg                      | 0.84      | 0.83   | 0.83     | 5581    |
| weighted avg                   | 0.84      | 0.84   | 0.84     | 5581    |

| Logistic Reg (L1) (threshold = 0.42) |           |        |          |         |
|--------------------------------------|-----------|--------|----------|---------|
|                                      | precision | recall | f1-score | support |
| Not Depressed                        | 0.86      | 0.73   | 0.79     | 2313    |
| Depressed                            | 0.83      | 0.92   | 0.87     | 3268    |
| accuracy                             |           |        | 0.84     | 5581    |
| macro avg                            | 0.84      | 0.82   | 0.83     | 5581    |
| weighted avg                         | 0.84      | 0.84   | 0.84     | 5581    |

| Balanced RF (threshold = 0.30) |           |        |          |         |
|--------------------------------|-----------|--------|----------|---------|
|                                | precision | recall | f1-score | support |
| Not Depressed                  | 0.83      | 0.74   | 0.78     | 2313    |
| Depressed                      | 0.83      | 0.90   | 0.86     | 3268    |
| accuracy                       |           |        | 0.83     | 5581    |
| macro avg                      | 0.83      | 0.82   | 0.82     | 5581    |
| weighted avg                   | 0.83      | 0.83   | 0.83     | 5581    |

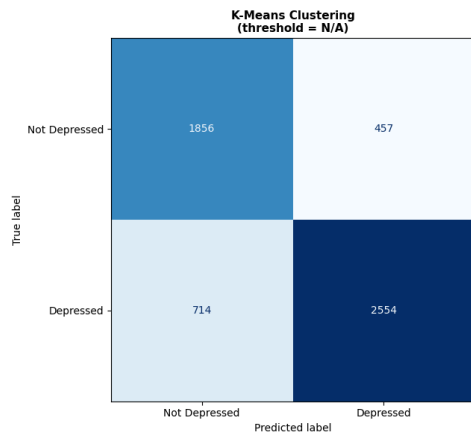
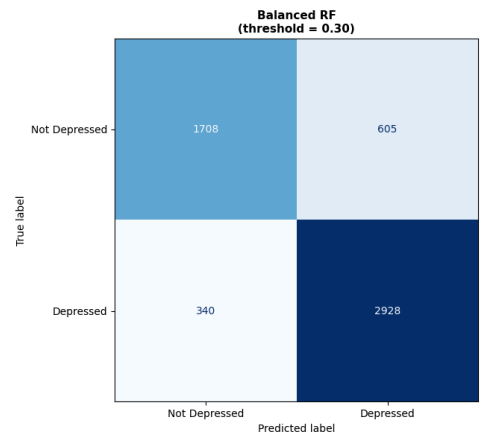
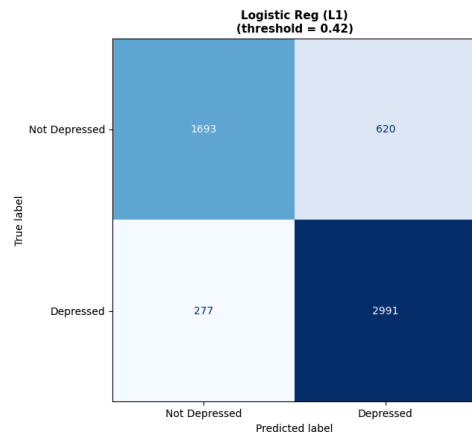
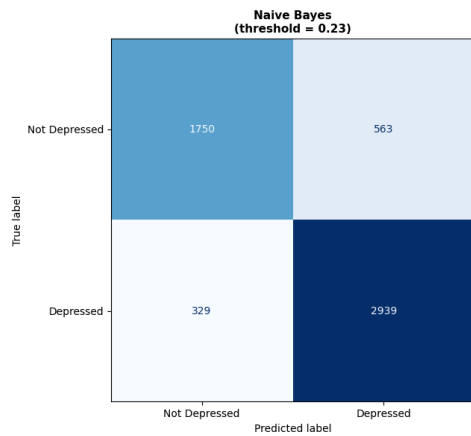
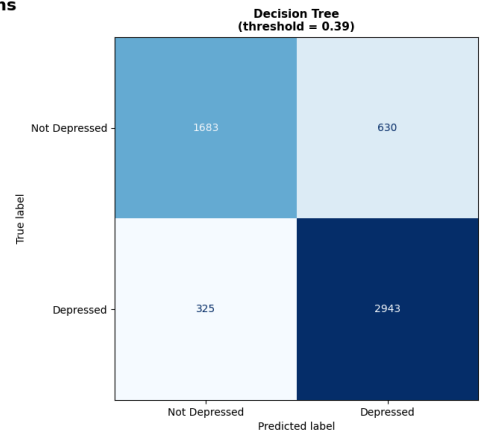
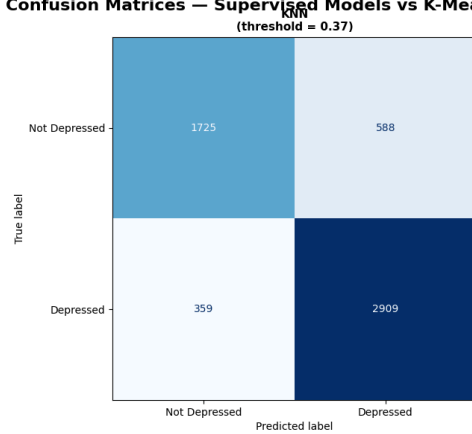
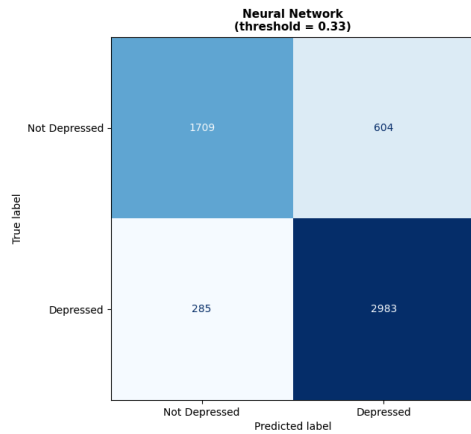
  

| K-Means Clustering (threshold = N/A (Clustering)) |           |        |          |         |
|---------------------------------------------------|-----------|--------|----------|---------|
|                                                   | precision | recall | f1-score | support |
| Not Depressed                                     | 0.72      | 0.80   | 0.76     | 2313    |
| Depressed                                         | 0.85      | 0.78   | 0.81     | 3268    |
| accuracy                                          |           |        | 0.79     | 5581    |
| macro avg                                         | 0.79      | 0.79   | 0.79     | 5581    |
| weighted avg                                      | 0.80      | 0.79   | 0.79     | 5581    |

## Confusion Matrices (all models)

```
1 fig, axes = plt.subplots(3, 3, figsize=(20, 18))
2 axes = axes.flatten()
3
4 all_plot_models = model_entries + [{"K-Means Clustering", km_test_pred, None, 0.50}]
5
6 for idx, (name, pred, prob, thr) in enumerate(all_plot_models):
7     ConfusionMatrixDisplay.from_predictions(
8         y_test, pred, ax=axes[idx], cmap='Blues',
9         colorbar=False, display_labels=['Not Depressed', 'Depressed'])
10
11     threshold_str = f"{thr:.2f}" if name != "K-Means Clustering" else "N/A"
12     axes[idx].set_title(f'{name}\n(threshold = {threshold_str})', fontsize=11, fontweight='bold')
13
14 for i in range(len(all_plot_models), len(axes)):
15     axes[i].axis('off')
16
17 plt.suptitle('Confusion Matrices - Supervised Models vs K-Means', fontsize=16, fontweight='bold')
18 plt.tight_layout(); plt.show()
```

## Confusion Matrices — Supervised Models vs K-Means

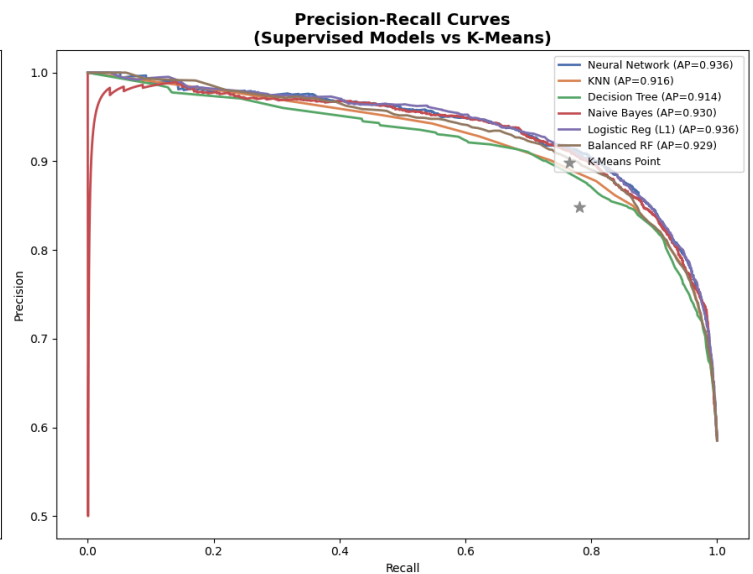
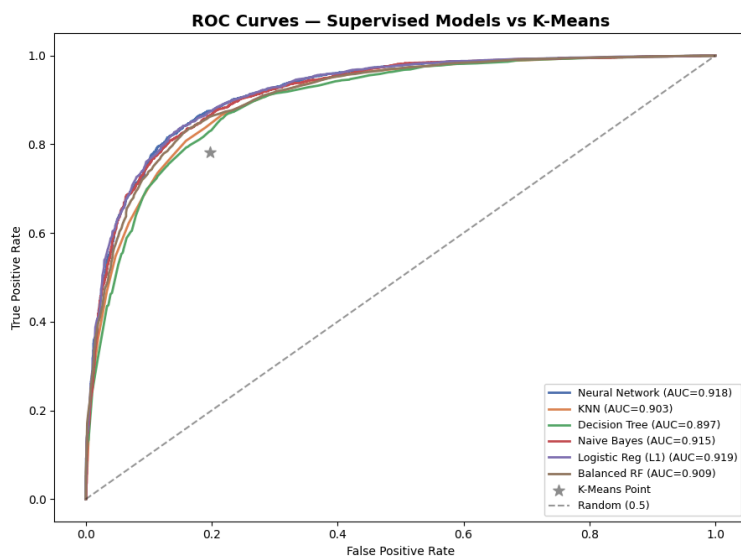


## ROC Curves + AUC Scores (all models)

```

1 fig, axes = plt.subplots(1, 2, figsize=(18, 7))
2 colors7 = ['#4C72B0', '#DD8452', '#55A868', '#C44E52', '#8172B2', '#937860', '#E74C3C']
3
4 # --- ROC Curves ---
5 for (name, pred, prob, thr), col in zip(model_entries, colors7):
6     fpr, tpr, _ = roc_curve(y_test, prob)
7     auc = roc_auc_score(y_test, prob)
8     axes[0].plot(fpr, tpr, label=f'{name} (AUC={auc:.3f})', color=col, linewidth=2)
9
10 # Add K-Means as a single point on ROC
11 km_fpr = 1 - recall_score(y_test, km_test_pred, pos_label=0)
12 km_tpr = recall_score(y_test, km_test_pred)
13 axes[0].scatter(km_fpr, km_tpr, color='#8c8c8c', s=100, marker='*', zorder=5, label='K-Means Point')
14
15 axes[0].plot([0, 1], [0, 1], 'k--', alpha=0.4, label='Random (0.5)')
16 axes[0].set_title('ROC Curves — Supervised Models vs K-Means', fontsize=14, fontweight='bold')
17 axes[0].set_xlabel('False Positive Rate')
18 axes[0].set_ylabel('True Positive Rate')
19 axes[0].legend(loc='lower right', fontsize=9)
20
21 # --- Precision-Recall Curves ---
22 for (name, pred, prob, thr), col in zip(model_entries, colors7):
23     pr, re, _ = precision_recall_curve(y_test, prob)
24     ap = average_precision_score(y_test, prob)
25     axes[1].plot(re, pr, label=f'{name} (AP={ap:.3f})', color=col, linewidth=2)
26
27 # Add K-Means as a single point on PR
28 km_prec = precision_score(y_test, km_test_pred)
29 km_rec = recall_score(y_test, km_test_pred)
30 axes[1].scatter(km_rec, km_prec, color='#8c8c8c', s=100, marker='*', zorder=5, label='K-Means Point')
31
32 axes[1].set_title('Precision-Recall Curves\n(Supervised Models vs K-Means)', fontsize=14, fontweight='bold')
33 axes[1].set_xlabel('Recall')
34 axes[1].set_ylabel('Precision')
35 axes[1].legend(loc='upper right', fontsize=9)
36
37 plt.tight_layout(); plt.show()

```



In mental health predictions, relying solely on "Accuracy" can be misleading due to class imbalance. We learned that we must look at a combination of metrics to evaluate true performance. Missing a depressed student (a False Negative) carries a high risk in real life. By comparing Precision and Recall, we learned which model is best at successfully identifying at-risk students without being overly sensitive (which would cause too many false alarms). The confusion matrices gave us a transparent look at exactly where each model makes mistakes (predicting 'Depressed' when they aren't, or vice versa). The ROC-AUC curves confirmed which models have the strongest underlying ability to distinguish between the two classes across different probability thresholds. By lining up six supervised models and evaluating their metric summaries, we learned which specific algorithm adapts best to the overlapping, noisy nature of student lifestyle and academic data.

## Performance Comparison (Accuracy, Precision, Recall, F1, AUC-ROC)

Based on the final test evaluation, the supervised models performed remarkably well:

- **Neural Network:** Emerged as the top performer with an Accuracy of 84.07%, F1-Score of 87.03%, Recall of 91.28%, and an AUC-ROC of 91.80%.
- **Logistic Regression:** Closely followed with an Accuracy of 83.93% and an AUC-ROC of 91.89%.
- **Naive Bayes & Balanced Random Forest:** Both maintained F1-scores above 86% with Recall scores over 89%.
- **K-Means Clustering:** The unsupervised model performed the poorest (Accuracy 79.02%, AUC 50.00%), confirming that labeled target data is required for this complex prediction.

## Conclusion

```
1 print('=' * 65)
2 print('          FINAL RESULTS – CSE422 Student Depression Prediction')
3 print('=' * 65)
4 print(res_df.to_string())
5 print(f'\nBest model: {res_df.index[0]} (F1 = {res_df["F1"].iloc[0]:.4f})')
```

```
=====
          FINAL RESULTS – CSE422 Student Depression Prediction
=====

```

|                    | Threshold | Accuracy | Precision | Recall | F1     | AUC-ROC |
|--------------------|-----------|----------|-----------|--------|--------|---------|
| Neural Network     | 0.34      | 0.8412   | 0.8344    | 0.9094 | 0.8703 | 0.9180  |
| Logistic Reg (L1)  | 0.42      | 0.8393   | 0.8283    | 0.9152 | 0.8696 | 0.9190  |
| Naive Bayes        | 0.23      | 0.8402   | 0.8392    | 0.8993 | 0.8682 | 0.9148  |
| Balanced RF        | 0.30      | 0.8314   | 0.8282    | 0.8984 | 0.8619 | 0.9096  |
| KNN                | 0.30      | 0.8233   | 0.8087    | 0.9146 | 0.8584 | 0.9013  |
| Decision Tree      | 0.33      | 0.8068   | 0.7841    | 0.9247 | 0.8486 | 0.8925  |
| K-Means Clustering | 0.50      | 0.7902   | 0.8482    | 0.7815 | 0.8135 | 0.5000  |

```
Best model: Neural Network (F1 = 0.8703)
```

The project successfully established an automated machine learning pipeline capable of evaluating student records to predict depression status. It shifts mental health monitoring from reactive manual surveys to a continuous, proactive, data-driven system, enabling universities to allocate counseling resources much more effectively.

We learned that academic pressure, financial stress, and sleep deprivation are the primary drivers of student depression. From a technical standpoint, handling class imbalances via SMOTE, properly scaling data within leakage-free pipelines, and heavily tuning supervised models (via Optuna) is vital for reliable predictions. Unsupervised algorithms struggle on this dataset due to high natural overlaps, proving that labeled target data and supervised models are the optimal approach for mental health screening.

Thus, throughout the dataset pre-processing, we faced challenges with missing values, feature scaling mismatches, and handling the class imbalance. By establishing a robust pipeline utilizing Stratified splitting, SMOTE, and Optuna tuning, we successfully optimized our models. Analyzing the aforementioned data, we can conclude that the **Neural Network** performed the best among all models. The exceptionally high Recall rate (over 91%) across our top models proves this system is highly effective at minimizing False Negatives, meaning it successfully identifies the vast majority of students who are actually at risk of depression.